Mini Project 13 - Membuka Test Set Sekali dan Membekukan Laporan Evaluasi

Tujuan

Tujuannya adalah mengevaluasi model klasik MP11 dan Tiny CNN MP12 pada test set beku MP9 tanpa mengubah model, menyimpan audit trail, dan menghasilkan status ready_for_mp14. Kedua kandidat diteruskan; keputusan deployment final belum dibuat.

A. Prasyarat

- Mini Project 12 selesai dan project_status.json bernilai ready_for_mp13.
- File models/mp11_classical_baseline.joblib dan models/mp12_tiny_cnn_state.pt tersedia.
- File features/dataset/mp10_features_final.npz serta features/logmel/mp09/*.npz tersedia.
- File splits/mp09_split.json masih identik dengan split yang dibekukan pada MP9.
- Python memiliki numpy, matplotlib, scikit-learn, joblib, dan PyTorch.

B. Langkah 1 - Validasi Gate sebelum Test Dibuka

Jalankan pemeriksaan berikut dari root folder proyek. Jangan lanjut jika salah satu assertion gagal.

```
python - <<'PY'
import json, pathlib
s=json.load(open('project_status.json'))
assert s['status']=='ready_for_mp13'
c=json.load(open('config/classifier_config.json'))
d=json.load(open('config/deep_learning_config.json'))
assert c['test_predictions_generated'] is False
assert d['test_predictions_generated'] is False
for p in ['models/mp11_classical_baseline.joblib',
        'models/mp12_tiny_cnn_state.pt',
        'splits/mp09_split.json',
        'features/dataset/mp10_features_final.npz']:
   assert pathlib.Path(p).exists(), p
print('PRE-TEST GATE VALID')
PY
```

Gate	Kondisi lulus	Mengapa penting
Status	ready_for_mp13	Menjamin urutan serial
MP11 test flag	False	Baseline belum menyentuh test
MP12 test flag	False	CNN belum menyentuh test
Split tersedia	mp09_split.json ada	Test identity tidak dibuat ulang
Model tersedia	Kedua file model ada	Evaluasi tidak retrain

Tabel 13.12. Gate wajib sebelum membuka final test.

C. Langkah 2 - Simpan Kode Python

Buat file scripts/mini_project_13.py, lalu salin kode berikut. Kode menghitung SHA-256 sebelum test, membuat prediction sekali, menyimpan laporan, memeriksa hash setelah test, dan mengubah status ke ready_for_mp14.

```
from pathlib import Path
import json, csv, hashlib, math
import numpy as np
import joblib
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.metrics import (
   accuracy_score, precision_recall_fscore_support, f1_score,
   balanced_accuracy_score, confusion_matrix
)

ROOT = Path('.')
TARGET_FRAMES = 96
ALPHA_Z = 1.96  # 95% Wilson interval
```

```
def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, indent=2), encoding='utf-8')


def sha256(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b''):
            h.update(chunk)
    return h.hexdigest()


def wilson_interval(k, n, z=1.96):
    if n == 0:
        return [0.0, 1.0]
    p = k / n
    den = 1 + z*z/n
    center = (p + z*z/(2*n)) / den
    half = z * math.sqrt((p*(1-p)/n) + (z*z/(4*n*n))) / den
    return [max(0.0, center-half), min(1.0, center+half)]


def model_metrics(y_true, y_pred, labels):
    p, r, f, s = precision_recall_fscore_support(
        y_true, y_pred, labels=labels, zero_division=0
    )
    acc = float(accuracy_score(y_true, y_pred))
    out = {
        'accuracy': acc,
        'balanced_accuracy': float(balanced_accuracy_score(y_true, y_pred)),
        'macro_precision': float(np.mean(p)),
        'macro_recall': float(np.mean(r)),
        'macro_f1': float(f1_score(y_true, y_pred, average='macro', zero_division=0)),
        'accuracy_wilson_95': wilson_interval(int(np.sum(np.asarray(y_true)==np.asarray(y_pred))), len(y_true), ALPHA_Z),
        'per_class': {},
        'confusion_matrix': confusion_matrix(y_true, y_pred, labels=labels).tolist(),
    }
    for i, c in enumerate(labels):
        out['per_class'][c] = {
            'precision': float(p[i]), 'recall': float(r[i]), 'f1': float(f[i]), 'support': int(s[i])
        }
    return out


# 0) Gate serial dan pre-test audit.
status_path = ROOT / 'project_status.json'
status = json.loads(status_path.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp13':
    raise RuntimeError(f"Status harus ready_for_mp13, ditemukan {status.get('status')!r}")

clf_cfg = json.loads((ROOT/'config/classifier_config.json').read_text(encoding='utf-8'))
dl_cfg = json.loads((ROOT/'config/deep_learning_config.json').read_text(encoding='utf-8'))
if clf_cfg.get('test_predictions_generated') or dl_cfg.get('test_predictions_generated'):
    raise RuntimeError('Gate gagal: salah satu model sudah pernah membuat test prediction sebelum MP13.')

split = json.loads((ROOT/'splits/mp09_split.json').read_text(encoding='utf-8'))
test_ids = list(split['test_ids'])
classes = list(clf_cfg['classes'])
if classes != list(dl_cfg['classes']):
    raise RuntimeError('Urutan kelas MP11 dan MP12 tidak identik.')

# Bekukan hash artefak SEBELUM test dibuka.
freeze_files = [
    ROOT/'models/mp11_classical_baseline.joblib',
    ROOT/'models/mp11_model_parameters.npz',
    ROOT/'config/classifier_config.json',
    ROOT/'models/mp12_tiny_cnn_state.pt',
    ROOT/'models/mp12_tiny_cnn_parameters.npz',
    ROOT/'config/deep_learning_config.json',
    ROOT/'config/feature_reduction_config.json',
    ROOT/'config/feature_config.json',
    ROOT/'splits/mp09_split.json',
]
for p in freeze_files:
    if not p.exists():
        raise FileNotFoundError(p)
pretest_hashes = {str(p): sha256(p) for p in freeze_files}

protocol = {
    'name': 'MP13 final frozen-test comparison',
    'test_open_policy': 'open_once_after_all_models_are_frozen',
    'test_ids': test_ids,
    'n_test': len(test_ids),
    'primary_metric': 'macro_f1',
```

```
    'secondary_metrics': ['accuracy','balanced_accuracy','macro_precision','macro_recall'],
    'confidence_interval': 'Wilson 95% interval for accuracy',
    'post_test_tuning_allowed': False,
    'deployment_model_selection_from_test_allowed': False,
    'deployment_candidates_forwarded': ['mp11_classical_baseline','mp12_tiny_cnn'],
    'device_based_selection_deferred_to': 'MP15',
    'warning': 'Test set is very small; metrics are demonstrative and statistically uncertain.'
}
save_json(ROOT/'config/evaluation_protocol.json', protocol)
save_json(ROOT/'results/metrics/mp13_pretest_artifact_hashes.json', pretest_hashes)

# 1) Buka test set untuk baseline klasik.
D = np.load(ROOT/'features/dataset/mp10_features_final.npz', allow_pickle=True)
X = D['X'].astype(np.float64)
y = D['y'].astype(str)
sample_ids = D['sample_ids'].astype(str)
test_mask = D['test_mask'].astype(bool)
idx_by_id = {sid:i for i,sid in enumerate(sample_ids)}
idx = np.array([idx_by_id[sid] for sid in test_ids], dtype=int)
if not np.all(test_mask[idx]):
    raise RuntimeError('Test ID tidak cocok dengan test_mask yang dibekukan.')
X_test = X[idx]
y_test = y[idx]

classical = joblib.load(ROOT/'models/mp11_classical_baseline.joblib')
pred_classical = classical.predict(X_test).astype(str)
# Margin hanya confidence proxy, bukan calibrated probability.
if hasattr(classical, 'decision_function'):
    scores = np.asarray(classical.decision_function(X_test))
    if scores.ndim == 1:
        margin_classical = np.abs(scores)
    else:
        ss = np.sort(scores, axis=1)
        margin_classical = ss[:,-1] - ss[:,-2]
else:
    margin_classical = np.full(len(y_test), np.nan)

# 2) Buka log-Mel test untuk CNN.
class TinyKWSNet(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1)),
        )
        self.classifier = nn.Linear(16, n_classes)
    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)

ckpt = torch.load(ROOT/'models/mp12_tiny_cnn_state.pt', map_location='cpu', weights_only=False)
if list(ckpt['classes']) != classes:
    raise RuntimeError('Kelas checkpoint CNN tidak cocok.')
mu = float(ckpt['normalization']['mean']); sigma = float(ckpt['normalization']['std'])
model = TinyKWSNet(len(classes)); model.load_state_dict(ckpt['state_dict']); model.eval()

logmels = []
labels_dl = []
for sid in test_ids:
    p = ROOT/'features/logmel/mp09'/f'{sid}.npz'
    d = np.load(p, allow_pickle=True)
    x = d['log_mel'].astype(np.float32)
    if x.shape[0] >= TARGET_FRAMES:
        x = x[:TARGET_FRAMES]
    else:
        fill = float(np.min(x)) if x.size else 0.0
        x = np.pad(x, ((0,TARGET_FRAMES-x.shape[0]),(0,0)), mode='constant', constant_values=fill)
    logmels.append((x-mu)/sigma)
    labels_dl.append(str(d['label']))
if labels_dl != y_test.tolist():
    raise RuntimeError('Label test log-Mel dan dataset klasik tidak identik pada urutan test_ids.')
Xdl = torch.from_numpy(np.stack(logmels)[:,None,:,:]).float()
with torch.no_grad():
    logits = model(Xdl)
    prob = torch.softmax(logits, dim=1).cpu().numpy()
    pred_idx = prob.argmax(axis=1)
pred_cnn = np.asarray([classes[i] for i in pred_idx], dtype=str)
conf_cnn = prob.max(axis=1)

# 3) Metrik final, TANPA mengubah model.
metrics_classical = model_metrics(y_test, pred_classical, classes)
metrics_cnn = model_metrics(y_test, pred_cnn, classes)
mp11_metrics = json.loads((ROOT/'results/metrics/mp11_classifier_metrics.json').read_text(encoding='utf-8'))
```

```
mp12_metrics = json.loads((ROOT/'results/metrics/mp12_deep_learning_metrics.json').read_text(encoding='utf-8'))
resource = {
  'classical': {
    'family': clf_cfg['chosen_family'],
    **mp11_metrics['family_results'][clf_cfg['chosen_family']]['resource_proxy'],
    'note': 'Classifier-only resource proxy; feature extraction cost is not included.'
  },
  'tiny_cnn': {
    'parameter_count': mp12_metrics['parameter_count'],
    'float32_parameter_bytes': mp12_metrics['float32_parameter_bytes'],
    'estimated_int8_parameter_bytes': mp12_metrics['estimated_int8_parameter_bytes'],
    'estimated_macs_per_inference': mp12_metrics['estimated_macs_per_inference'],
    'note': 'Architecture proxy; measured latency/RAM/energy are deferred to MP15.'
  }
}
report = {
  'test_opened': True,
  'test_opened_once_in_running_project': True,
  'n_test': int(len(y_test)),
  'classes': classes,
  'sample_ids': test_ids,
  'classical': metrics_classical,
  'tiny_cnn': metrics_cnn,
  'resource_proxy': resource,
  'post_test_tuning_performed': False,
  'deployment_selection_from_test_performed': False,
  'deployment_candidates_forwarded': ['mp11_classical_baseline','mp12_tiny_cnn'],
  'interpretation_warning': 'With four test samples, point estimates have very wide uncertainty. Do not claim perfect generalization from a 100% score.',
}
save_json(ROOT/'results/metrics/mp13_final_test_metrics.json', report)

# Per-sample audit trail.
pred_dir = ROOT/'results/predictions'; pred_dir.mkdir(parents=True, exist_ok=True)
with open(pred_dir/'mp13_test_predictions.csv','w',newline='',encoding='utf-8') as f:
  w = csv.writer(f)
  w.writerow(['sample_id','true_label','classical_prediction','classical_margin_proxy','cnn_prediction','cnn_max_softmax'])
  for i,sid in enumerate(test_ids):
    w.writerow([sid,y_test[i],pred_classical[i],float(margin_classical[i]),pred_cnn[i],float(conf_cnn[i])])

# 4) Gambar evaluasi.
fig_dir = ROOT/'results/figures'; fig_dir.mkdir(parents=True, exist_ok=True)
cm1 = np.asarray(metrics_classical['confusion_matrix']); cm2 = np.asarray(metrics_cnn['confusion_matrix'])
fig, axes = plt.subplots(1,2,figsize=(8.5,3.6))
for ax,cm,title in zip(axes,[cm1,cm2],['MP11 classical','MP12 Tiny CNN']):
  im=ax.imshow(cm,cmap='Blues',vmin=0,vmax=max(1,cm.max()))
  ax.set_xticks(range(len(classes))); ax.set_yticks(range(len(classes)))
  ax.set_xticklabels(classes,rotation=35,ha='right'); ax.set_yticklabels(classes)
  ax.set_xlabel('Prediksi'); ax.set_ylabel('Aktual'); ax.set_title(title)
  for i in range(len(classes)):
    for j in range(len(classes)):
      ax.text(j,i,str(cm[i,j]),ha='center',va='center')
fig.tight_layout(); fig.savefig(fig_dir/'mp13_test_confusion_matrices.png',dpi=180); plt.close(fig)

names=['Accuracy','Balanced Acc.','Macro Precision','Macro Recall','Macro F1']
keys=['accuracy','balanced_accuracy','macro_precision','macro_recall','macro_f1']
x=np.arange(len(names)); width=.36
fig,ax=plt.subplots(figsize=(8.4,4.1))
ax.bar(x-width/2,[metrics_classical[k] for k in keys],width,label='Classical')
ax.bar(x+width/2,[metrics_cnn[k] for k in keys],width,label='Tiny CNN')
ax.set_ylim(0,1.08); ax.set_xticks(x); ax.set_xticklabels(names,rotation=20,ha='right'); ax.set_ylabel('Skor'); ax.legend(); ax.grid(axis='y',alpha=.2)
fig.tight_layout(); fig.savefig(fig_dir/'mp13_test_metric_comparison.png',dpi=180); plt.close(fig)

fig,ax=plt.subplots(figsize=(7.6,4.0))
labels=['Classical\nops proxy','Tiny CNN\nMAC proxy']
vals=[resource['classical']['estimated_ops_per_sample'],resource['tiny_cnn']['estimated_macs_per_inference']]
ax.bar(labels,vals); ax.set_yscale('log'); ax.set_ylabel('Operasi per inference (log scale)'); ax.grid(axis='y',alpha=.2)
for i,v in enumerate(vals): ax.text(i,v*1.15,f'{v:,}',ha='center')
fig.tight_layout(); fig.savefig(fig_dir/'mp13_resource_proxy.png',dpi=180); plt.close(fig)

# 5) Hash sesudah test untuk memastikan model/config input tidak berubah.
posttest_hashes = {str(p): sha256(p) for p in freeze_files}
if posttest_hashes != pretest_hashes:
  raise RuntimeError('Artefak model/config berubah selama evaluasi test.')
save_json(ROOT/'results/metrics/mp13_posttest_artifact_hashes.json', posttest_hashes)

# 6) Update status serial. Kedua kandidat dibawa ke MP14/MP15.
status['status'] = 'ready_for_mp14'
status['last_completed_mini_project'] = 13
status['mp13'] = {
  'evaluation_protocol': 'config/evaluation_protocol.json',
  'final_test_metrics': 'results/metrics/mp13_final_test_metrics.json',
  'test_predictions': 'results/predictions/mp13_test_predictions.csv',
  'pretest_hashes': 'results/metrics/mp13_pretest_artifact_hashes.json',
  'posttest_hashes': 'results/metrics/mp13_posttest_artifact_hashes.json',
  'test_opened': True,
  'post_test_tuning_performed': False,
  'deployment_candidates': ['models/mp11_classical_baseline.joblib','models/mp12_tiny_cnn_state.pt'],
```

```
  'deployment_selection_deferred_to': 'MP15'
}
save_json(status_path, status)

print('MP13 VALID: final frozen test evaluated once')
print('classical macro-F1 =', round(metrics_classical['macro_f1'],4))
print('tiny CNN macro-F1 =', round(metrics_cnn['macro_f1'],4))
print('classical accuracy 95% Wilson =', [round(v,4) for v in metrics_classical['accuracy_wilson_95']])
print('tiny CNN accuracy 95% Wilson =', [round(v,4) for v in metrics_cnn['accuracy_wilson_95']])
print('post-test tuning = False')
print('status = ready_for_mp14')
```

D. Langkah 3 - Jalankan Evaluasi Final

```
python scripts/mini_project_13.py
```

Output terminal yang benar harus berakhir dengan MP13 VALID, post-test tuning = False, dan status = ready_for_mp14. Nilai macro-F1 mahasiswa dapat berbeda dari runtime test buku.

E. Langkah 4 - Baca Hasil tanpa Retuning

```
python - <<'PY'
import json
r=json.load(open('results/metrics/mp13_final_test_metrics.json'))
for name in ['classical','tiny_cnn']:
  m=r[name]
  print(name, 'acc=', round(m['accuracy'],3), 'macroF1=', round(m['macro_f1'],3),
      'Wilson95=', [round(x,3) for x in m['accuracy_wilson_95']])
print('post-test tuning =', r['post_test_tuning_performed'])
PY
```

Aturan setelah melihat hasil

Jangan menjalankan kembali MP10, MP11, atau MP12 dengan parameter baru hanya karena hasil test tidak sesuai harapan. Jika ditemukan failure mode, catat sebagai temuan dan rancang pengumpulan data/eksperimen baru di luar test set yang sudah dibuka.

F. Langkah 5 - Audit Prediksi dan Artefak

```
python - <<'PY'
import csv, json
print(open('results/predictions/mp13_test_predictions.csv').read())
a=json.load(open('results/metrics/mp13_pretest_artifact_hashes.json'))
b=json.load(open('results/metrics/mp13_posttest_artifact_hashes.json'))
assert a==b
print('HASH VALID: model/config tidak berubah selama final test')
PY
```

Artefak	Fungsi	Dibawa ke MP14/15
config/evaluation_protocol.json	Kontrak evaluasi final	Ya, sebagai audit
mp13_final_test_metrics.json	Metrik final kedua kandidat	Ya
mp13_test_predictions.csv	Audit per sampel	Ya
mp13_pretest_artifact_hashes.json	Fingerprint sebelum test	Ya
mp13_posttest_artifact_hashes.json	Fingerprint sesudah test	Ya
mp11_classical_baseline.joblib	Kandidat backend klasik	Ya
mp12_tiny_cnn_state.pt	Kandidat backend CNN	Ya

Tabel 13.13. Artefak wajib yang diteruskan setelah Mini Project 13.

G. Langkah 6 - Validasi Handoff ke Mini Project 14

```
python - <<'PY'
import json, pathlib
s=json.load(open('project_status.json'))
assert s['status']=='ready_for_mp14'
assert s['mp13']['test_opened'] is True
assert s['mp13']['post_test_tuning_performed'] is False
for p in ['config/evaluation_protocol.json',
      'results/metrics/mp13_final_test_metrics.json',
      'results/predictions/mp13_test_predictions.csv']:
  assert pathlib.Path(p).exists(), p
print('MP13 VALID: siap Mini Project 14')
PY
```

Handoff Serial MP13 -> MP14

Mini Project 14 tidak melakukan training atau membuka split baru. Ia menggunakan requirement sistem, preprocessing yang sudah baku, dan kedua backend kandidat untuk merancang aliran real-time: acquisition, buffering, feature extraction, inference, post-processing, dan timing budget. Benchmark device dan kuantisasi tetap dilakukan pada MP15.

Referensi

1. Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. Journal of Machine Learning Technologies, 2(1), 37-63.

2. Sokolova, M., & Lapalme, G. (2009). A Systematic Analysis of Performance Measures for Classification Tasks. Information Processing & Management, 45(4), 427-437.

3. Fawcett, T. (2006). An Introduction to ROC Analysis. Pattern Recognition Letters, 27(8), 861-874.

4. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432.

5. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. Proceedings of ICML.

6. Brier, G. W. (1950). Verification of Forecasts Expressed in Terms of Probability. Monthly Weather Review, 78(1), 1-3.

7. Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association, 22(158), 209-212.

8. Flach, P. (2012). Machine Learning: The Art and Science of Algorithms that Make Sense of Data. Cambridge University Press.

9. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

10. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2020). Efficient Processing of Deep Neural Networks. Morgan & Claypool.